

BACKEND DESIGN DOCUMENT

SimuFlow

Interactive System Design Visualiser

v1.0 · March 2026

FastAPI + Python
API Layer

Supabase
Auth + DB

Postgres + RLS
Data Security

1. Backend Philosophy

SimuFlow's backend is intentionally thin. The simulation engine, chaos logic, and all real-time computation live in the frontend Web Worker. The backend does three things: verify identity, persist diagrams, and serve presets. It does not simulate anything.

Core rule: if a feature can live in the frontend without compromising security or data integrity, it lives in the frontend. The backend is not a simulation server. It is an identity, persistence, and API layer.

1.1 What the Backend Does

- Authentication — delegated entirely to Supabase Auth; FastAPI only verifies JWTs
- Diagram CRUD — save, load, update, delete user diagrams as TopologySchema JSON in Postgres
- Share links — generate and serve read-only diagram snapshots via unique tokens
- Fork — clone a public shared diagram into the requesting user's account
- Preset serving — return the 4 curated architecture blueprints

1.2 What the Backend Does NOT Do

- Run simulations — Web Worker only
- Store simulation frames, tick history, or metric snapshots — ephemeral browser state
- Real-time connections — no WebSocket server; frontend communicates via REST only
- Chaos scenario logic — chaos runs in the frontend chaosModifier, not here

2. Technology Stack

Package	Version	Purpose	Rationale
fastapi	^0.110	Web framework	Native async, Pydantic v2 validation, auto OpenAPI at /docs
uvicorn	^0.29	ASGI server	Production-grade with uvloop
pydantic	^2	Validation	Request/response schema validation — mirrors topology.ts types
supabase-py	^2	DB + auth client	Supabase admin client for server-side operations
python-jose	^3.3	JWT verification	Verify Supabase-issued JWTs in get_current_user dependency
python-dotenv	^1.0	Config	Environment variable loading
pytest	^8	Testing	Test framework
pytest-asyncio	^0.23	Async tests	FastAPI route testing with async client
httpx	^0.27	Test client	Async HTTP client for integration tests

3. Project Structure

Path	Contents
app/main.py	FastAPI app instance, CORS config, router registration, lifespan hooks
app/api/v1/	Route handlers: diagrams.py, presets.py, share.py, health.py
app/api/deps.py	Shared FastAPI dependencies: get_current_user, get_supabase
app/models/	Pydantic models — mirror of topology.ts types. See §8 for full mapping.
app/services/	Business logic: DiagramService, PresetService, ShareService
app/db/	Supabase client factory, table name constants
app/core/config.py	Settings loaded from environment variables via pydantic-settings
app/core/security.py	JWT decode + Supabase user verification, AuthUser model
presets/	JSON files: web_app.json, cached_web_app.json, microservices.json, queue_system.json
supabase/migrations/	SQL migration files — run via Supabase CLI
tests/	Pytest files mirroring app/ structure
.env.example	Template of all required environment variables
Dockerfile	python:3.12-slim base, production container

4. Database Design

4.1 Tables

diagrams

Column	Type	Constraints	Description
id	uuid	PK, default gen_random_uuid()	Diagram identifier
user_id	uuid	FK auth.users, NOT NULL	Owner — Supabase auth user
name	text	NOT NULL, default 'Untitled'	Display name
topology	jsonb	NOT NULL	Full TopologySchema JSON — see §8 for schema
is_public	boolean	default false	Whether diagram has an active share link
share_token	text	unique, nullable	Random URL-safe token for read-only share URL
fork_count	integer	default 0	How many times this diagram has been forked
created_at	timestamptz	default now()	Creation timestamp
updated_at	timestamptz	default now(), trigger	Updated by trigger on any row change

presets

Column	Type	Description
id	uuid	PK
slug	text	Unique key: web_app, cached_web_app, microservices, queue_system
name	text	Display name: 'Basic Web App', 'Cached Web App', etc.
description	text	One-line description shown in preset picker UI
topology	jsonb	Full TopologySchema JSON for this blueprint
category	text	fundamentals patterns
sort_order	integer	Display order in preset picker
is_active	boolean	Toggle to hide without deleting

4.2 Row Level Security

All RLS policies are defined in SQL migrations — never in application code. The FastAPI service does not filter by `user_id` in query logic. RLS enforces isolation at the Postgres layer, making it impossible to accidentally expose another user's data even if application code has a bug.

Table	Operation	Policy
diagrams	SELECT	<code>user_id = auth.uid()</code> OR <code>is_public = true</code>
diagrams	INSERT	<code>user_id = auth.uid()</code>
diagrams	UPDATE	<code>user_id = auth.uid()</code>
diagrams	DELETE	<code>user_id = auth.uid()</code>
presets	SELECT	Authenticated users only — read-only catalogue
presets	INSERT / UPDATE / DELETE	Service role only — never via API

5. API Routes

Method	Path	Auth	Description
GET	/health	None	Health check — 200 + { status: ok, version }
GET	/api/v1/presets	JWT	List all active presets ordered by sort_order
GET	/api/v1/presets/{slug}	JWT	Single preset by slug — 404 if not found
GET	/api/v1/diagrams	JWT	List user's diagrams — paginated, ordered by updated_at desc
POST	/api/v1/diagrams	JWT	Create new diagram
GET	/api/v1/diagrams/{id}	JWT	Get diagram by ID — RLS blocks other users' private diagrams
PUT	/api/v1/diagrams/{id}	JWT	Partial update — name and/or topology
DELETE	/api/v1/diagrams/{id}	JWT	Delete diagram
POST	/api/v1/diagrams/{id}/share	JWT	Generate share token, set is_public=true
DELETE	/api/v1/diagrams/{id}/share	JWT	Revoke share link — clears token, sets is_public=false
GET	/api/v1/shared/{token}	None	Get public diagram by share token
POST	/api/v1/shared/{token}/fork	JWT	Fork public diagram into authenticated user's account

5.1 Request / Response Models

Full Pydantic model definitions are in §8 alongside the TypeScript mirrors. Below is the shape reference.

Endpoint	Request body	Response body
POST /diagrams	{ name?, topology }	DiagramResponse
PUT /diagrams/{id}	{ name?, topology? }	DiagramResponse
GET /diagrams	—	DiagramListResponse
GET /diagrams/{id}	—	DiagramResponse
POST /diagrams/{id}/share	—	ShareResponse
GET /shared/{token}	—	DiagramResponse (public fields only)
POST /shared/{token}/fork	—	ForkResponse
GET /presets	—	PresetBlueprint[]

6. Authentication

6.1 Auth Flow

1. User signs in via Supabase Auth (email/password or OAuth) in the frontend
2. Supabase issues a JWT access token stored in the frontend Supabase client
3. Frontend includes token as Authorization: Bearer <token> on all API requests
4. FastAPI `get_current_user` dependency calls `supabase.auth.get_user(token)`
5. Valid → extracts `user.id`, returns `AuthUser` model to route handler
6. Invalid/expired → raises `HTTPException(401, 'Invalid or expired token')`

Never decode the JWT manually in route handlers. Always use `supabase.auth.get_user()` — it validates against Supabase's live keys and correctly handles token revocation.

6.2 CORS

- Allowed origins: `FRONTEND_URL` env var
- Allowed methods: GET, POST, PUT, DELETE, OPTIONS
- Allowed headers: Authorization, Content-Type
- `allow_credentials`: True — required for JWT auth headers

7. Services Layer

7.1 DiagramService

Method	Parameters	Returns	Notes
create	user_id, name, topology	DiagramResponse	Inserts row, returns full diagram
get	diagram_id, user_id	DiagramResponse None	RLS enforces ownership
list	user_id, page, page_size	DiagramListResponse	Ordered by updated_at desc, summaries only
update	diagram_id, user_id, patch	DiagramResponse	Partial update — only set fields applied
delete	diagram_id, user_id	bool	True if deleted, False if not found

7.2 ShareService

Method	Parameters	Returns	Notes
generate_share	diagram_id, user_id	ShareResponse	Generates token via secrets.token_urlsafe(16), sets is_public=true
revoke_share	diagram_id, user_id	bool	Clears share_token, sets is_public=false
get_by_token	share_token	DiagramResponse None	No auth — public read
fork	share_token, user_id	ForkResponse	Reads public diagram, creates copy in user's account

7.3 PresetService

- list_presets() — all active presets ordered by sort_order
- get_by_slug(slug) — single preset or raises 404
- Results cached in app state at startup (lifespan event) — no DB call per request

8. Pydantic Models — topology.ts Mirror

Every TypeScript interface in `src/types/topology.ts` has a corresponding Pydantic model in `app/models/`. This section is the canonical mapping between the two. When a field changes in `topology.ts`, the corresponding Pydantic model must be updated to match, and vice versa.

Naming convention: TypeScript uses camelCase, Python uses snake_case. The Pydantic base class uses `alias_generator=to_camel` so the API outputs camelCase JSON automatically. The frontend never transforms field names — what TypeScript defines is what the API returns.

8.1 Pydantic Base Config

```
// All Pydantic models inherit from this base
from pydantic import BaseModel, ConfigDict
from pydantic.alias_generators import to_camel

class SimuFlowBase(BaseModel):
    model_config = ConfigDict(
        populate_by_name=True,    # accept both snake and camel
        alias_generator=to_camel, # output camelCase JSON
    )
```

8.2 Enums

TypeScript enum	Python enum	Values
NodeType	<code>class NodeType(str, Enum)</code>	client, load_balancer, api_server, cache, database, queue, cdn, microservice
LBStrategy	<code>class LBStrategy(str, Enum)</code>	round_robin, least_connections, random
NodeHealth	<code>class NodeHealth(str, Enum)</code>	idle, healthy, stressed, bottleneck, failed
ChaosScenarioId	<code>class ChaosScenarioId(str, Enum)</code>	DB_FAILOVER, CACHE_STAMPEDE, NET_PARTITION, TRAFFIC_SPIKE, CASCADE, SLOW_DEP

8.3 Node Config Models

TypeScript interface	Pydantic model	Key fields (snake_case in Python)
BaseNodeConfig	<code>BaseNodeConfig(SimuFlowBase)</code>	capacity, latency_ms, failure_rate, timeout_ms

ClientConfig	ClientConfig(SimuFlowBase)	rps, burst
LoadBalancerConfig	LoadBalancerConfig(BaseNodeConfig)	strategy: LBStrategy, replicas
CacheConfig	CacheConfig(BaseNodeConfig)	hit_rate
QueueConfig	QueueConfig(SimuFlowBase)	delay_ms, max_depth
NodeConfig (union)	NodeConfig = Annotated[Union[...], Discriminator('node_type')]	Pydantic v2 discriminated union on node_type field

8.4 Topology Models

TypeScript	Pydantic	Notes
CanvasPosition	CanvasPosition(SimuFlowBase)	x: float, y: float
CanvasViewport	CanvasViewport(SimuFlowBase)	x: float, y: float, zoom: float
SimNode	NodeDef(SimuFlowBase)	id, label, position: CanvasPosition — extends NodeConfig union
SimEdge	EdgeDef(SimuFlowBase)	id, source_id, target_id, label: str None
TopologySchema	TopologySchema(SimuFlowBase)	version: str, nodes: list[NodeDef], edges: list[EdgeDef], viewport: CanvasViewport

8.5 API Response Models

TypeScript interface	Pydantic model	Key fields
DiagramSummary	DiagramSummary(SimuFlowBase)	id, name, is_public, created_at, updated_at
DiagramResponse	DiagramResponse(DiagramSummary)	topology: TopologySchema, share_token: str None, fork_count: int
DiagramListResponse	DiagramListResponse(SimuFlowBase)	items: list[DiagramSummary], total, page, page_size
ShareResponse	ShareResponse(SimuFlowBase)	share_url: str, share_token: str, is_public: Literal[True]
ForkResponse	ForkResponse(SimuFlowBase)	diagram_id: str, name: str
PresetBlueprint	PresetBlueprint(SimuFlowBase)	slug, name, description, category, topology: TopologySchema

8.6 Validation on Write

The topology JSONB column is validated by Pydantic before every write. If a node has `nodeType: 'cache'` but is missing `hitRate`, the API returns 422 before touching the database.

```
// POST /api/v1/diagrams - FastAPI validates body against CreateDiagramRequest
class CreateDiagramRequest(SimuFlowBase):
    name: str = "Untitled Diagram"
    topology: TopologySchema # Full validation - NodeDef discriminated union
checked

// PUT /api/v1/diagrams/{id} - partial update
class UpdateDiagramRequest(SimuFlowBase):
    name: str | None = None
    topology: TopologySchema | None = None
```

9. Error Handling

Status	When	Response body
400 Bad Request	Invalid topology JSON, schema validation failure	{ detail: string }
401 Unauthorized	Missing, malformed, or expired JWT	{ detail: 'Invalid or expired token' }
403 Forbidden	User attempts to modify another user's diagram	{ detail: 'Access denied' }
404 Not Found	Diagram, preset, or share token not found	{ detail: 'Diagram not found' }
422 Unprocessable	Pydantic validation error (auto-generated by FastAPI)	{ detail: [{ loc, msg, type }] }
500 Internal Error	Unexpected server error, Supabase unavailable	{ detail: 'Internal server error' }

- Global `exception_handler`: catches generic `Exception` — logs traceback server-side, returns generic 500
- Supabase client errors caught in service layer — re-raised as `HTTPException` with correct status
- Never expose raw Supabase error messages, stack traces, or internal IDs to the client

10. Environment & Deployment

10.1 Environment Variables

Variable	Required	Description
SUPABASE_URL	Yes	Supabase project URL
SUPABASE_SERVICE_KEY	Yes	Service role key for server-side DB access — never exposed to client
SUPABASE_JWT_SECRET	Yes	JWT secret for token verification
FRONTEND_URL	Yes	Allowed CORS origin — e.g. https://simuflow.app
ENV	Yes	development staging production
LOG_LEVEL	No	DEBUG INFO WARNING ERROR — default INFO

10.2 Supabase Migrations

Migration	Contents
001_create_diagrams.sql	diagrams table + all RLS policies + updated_at trigger
002_create_presets.sql	presets table + seed data for all 4 blueprint presets
003_add_share_fields.sql	is_public, share_token, fork_count columns on diagrams

Run migrations via Supabase CLI: supabase db push (local dev). Never apply schema changes manually in the Supabase dashboard — all changes must be reproducible via migrations.

11. Testing Strategy

11.1 Unit Tests

- DiagramService: create, read, list, update, delete — mock Supabase client
- ShareService: token generation, revocation, fork — verify topology copy correctness
- PresetService: list ordering, slug lookup, 404 on missing slug
- Security: JWT extraction, invalid token rejection, missing Authorization header

11.2 Integration Tests

- All routes tested via httpx.AsyncClient with FastAPI TestClient
- Supabase client mocked at service layer — no real DB calls in CI
- Auth dependency overridden to return fixed AuthUser in test context
- RLS coverage: get_diagram with mismatched user_id returns 404 (never confirm existence to attacker)

11.3 Test Files

File	Coverage
tests/test_diagrams.py	CRUD routes, pagination, ownership, 404 behaviour
tests/test_share.py	Share generation, revocation, public access, fork
tests/test_presets.py	Preset listing, slug lookup, inactive preset not returned
tests/test_auth.py	Valid JWT, expired JWT, missing header, malformed header
tests/test_models.py	Pydantic model validation — NodeConfig discriminated union, TopologySchema
tests/test_health.py	Health check response shape

12. Build Milestones

Milestone	Scope	Est.
M1 — Scaffold	FastAPI app, CORS, uvicorn, health endpoint, config, .env setup	1 day
M2 — Auth	Supabase client, get_current_user dependency, JWT verification, auth tests	2 days
M3 — Migrations	Migrations 001+002+003, RLS policies, preset seed data, Supabase CLI setup	2 days
M4 — Diagrams API	CRUD routes, DiagramService, Pydantic models, pagination, integration tests	3 days
M5 — Share & Fork	ShareService, share/revoke/fork routes, public diagram endpoint, tests	2 days
M6 — Presets API	PresetService, preset endpoints, in-memory cache on startup, tests	1 day
M7 — Deploy	Dockerfile, Railway/Render setup, env vars, health check, staging smoke test	1 day
Total: ~12 engineering days. M2 through M4 are sequential. M5 and M6 can run in parallel. The backend is intentionally thin — the simulation engine is the product, not the API layer.		